

Coding.Space

Autonomous code production pipeline and vibe coding platform for all languages — from web apps to C++ and simulations.

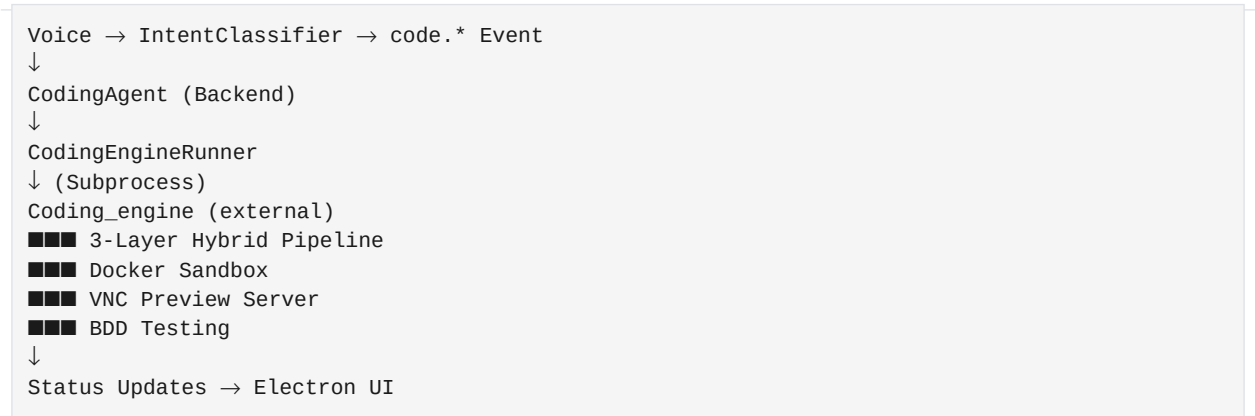
Overview

Coding.Space is Vibemind's software development platform with two modes:

- 1. **Autonomous Pipeline:** Based on a requirements stack, the Coding_engine (Society of Mind with 40+ agents) autonomously generates complete projects — including Docker sandbox, testing, and VNC preview.
- 2. **Vibe Coding:** Similar to Lovable, but language-agnostic — users describe via voice what they want and see the code emerge in real-time. Supports not just web technologies, but also C++, simulations, and other software languages.

The architecture clearly separates: VibeMind controls and monitors, the external Coding_engine generates the code.

Architecture



Backend Agent

File: `python/spaces/coding/agents/coding_agent.py`

Event	Tool Function	Description
<code>code.generate</code>	<code>generate_code</code>	Start code generation
<code>code.status</code>	<code>get_generation_status</code>	Query generation status
<code>code.cancel</code>	<code>cancel_generation</code>	Cancel generation
<code>code.list</code>	<code>list_generated_projects</code>	List projects
<code>code.preview.start</code>	<code>start_preview</code>	Start VNC preview
<code>code.preview.stop</code>	<code>stop_preview</code>	Stop preview

Event	Tool Function	Description
<code>idea.to_project</code>	<code>idea_to_project_sync</code>	Convert Bubble idea to project
<code>code.modify</code>	<code>modify_code_sync</code>	Modify code via voice

Key Components

Component	File	Purpose
Backend Agent	<code>agents/coding_agent.py</code>	Event routing to tools
Engine Runner	<code>engine/coding_engine_runner.py</code> (690 lines)	Subprocess bridge to Coding_engine
Project Discovery	<code>engine/project_discovery.py</code>	Project detection
Coding Tools	<code>tools/coding_tools.py</code> (587 lines)	Main tool implementations
Adapted Tools	<code>tools/adapted_coding_tools.py</code>	Typed wrappers
Voice Tools	<code>tools/voice_coding_tools.py</code>	Async voice wrappers
Broadcast Agent	<code>broadcast/coding_broadcast_agent.py</code>	Electron UI updates
Coding_engine	<code>Coding_engine/</code> (Git submodule)	External code generator

CodingEngineRunner

The runner (`engine/coding_engine_runner.py`) manages the external Coding_engine as a subprocess:

- **VNC Port Allocation:** Unique ports per job starting from BASE_VNC_PORT (6080)
- **Requirements Generation:** Creates `requirements.json` from voice input
- **Real-time Status Streaming:** Parses JSON + text progress updates
- **Quality Monitoring:** Polls `self_critique_report.json` during generation
- **4 Convergence Modes:** `autonomous`, `strict`, `relaxed`, `fast`
- **Job Lifecycle:** pending → generating → converging → testing → completed/failed

Subprocess Flags:

```
python run_engine.py <requirements.json> \  
--autonomous --continuous-sandbox --external-sandbox \  
--enable-vnc --vnc-port 6080 --enable-validation \  
--output-dir <path> --json-progress --parallel 10
```

VNC Preview System

Fully implemented with port management:

- `start_preview()` — Allocates port, starts VNC server, stores URL in DB
- `stop_preview()` — Terminates server, frees port

- `_find_available_port()` — Prevents port collisions
- DB fields: `vnc_port` `INTEGER`, `preview_url` `TEXT` in `projects` table

Testing

The external `Coding_engine` contains BDD testing infrastructure:

- 23 Gherkin feature files (user stories)
- Pytest step definitions with factories and fixtures
- `seed_data.sql` for test databases
- `--enable-validation` flag for automatic tests

Note: Test results are processed by the external subprocess. Voice-driven test execution and result display in Electron are not yet implemented.

Current Status

Implemented

- Voice-driven code generation via intent classification
- 8 event types with complete tool mapping
- `CodingEngineRunner` as subprocess bridge
- VNC preview system with multi-project port management
- Real-time status streaming to Electron UI
- Job lifecycle management (start, cancel, status)
- Quality monitoring via `self_critique_report.json`
- Docker sandbox integration (flags to engine)
- Project tracking in SQLite (`projects` table)
- Idea-to-project conversion from Bubbles

Not Implemented

- Test results not displayed in Electron UI
- No voice-driven individual test execution
- No code review gate in VibeMind (external in `Coding_engine`)
- Production deployment pipeline beyond preview
- UI/UX design-to-code conversion

Roadmap

- Display test results and quality reports in Electron UI
- Voice-driven test execution and debugging
- Production deployment infrastructure beyond preview

- Specialized code generation for domain-specific problems
- Code review system with automated quality checks
- Progressive refinement (iterative improvement based on feedback)
- Containerized environment generation for complex deployments

Ecosystem Fit

Coding.Space transforms intent into production code. Ideas.Space articulates what should be built, and Coding.Space generates it. Rowboat.Space provides business logic and data requirements. Software Design.Space validates architectural decisions. Desktop.Space validates that deployed applications work correctly. The Brain.Space informs architectural decisions through pattern recognition.